2.10 (2) — Триггеры. inserted и deleted

Конспект

Триггер автоматически запускается при событии. DML-триггеры реагируют на INSERT, UPDATE и DELETE. AFTER выполняется после операции и проверок ограничений, INSTEAD OF выполняет свою логику вместо операции.

inserted содержит новые строки, deleted - старые. При INSERT заполнена inserted, при DELETE - deleted, при UPDATE доступны обе. Старую и новую версии сопоставляют по ключу.

Триггер запускается для инструкции и обрабатывает набор строк. Поэтому аудит записывается через INSERT ... SELECT, а не через одну скалярную переменную.

Триггеры используются для аудита и сложных правил. Простое ограничение лучше задавать CHECK. THROW позволяет сообщить об ошибке, и нарушающая операция откатывается.

Ответы на вопросы для самопроверки

1. Что такое триггер?
Объект базы, автоматически выполняющийся при заданном событии.

2. Почему триггер нельзя вызвать обычной командой EXEC?
Он связан с событием и запускается сервером автоматически, а не как процедура через EXEC.

3. Какие DML события могут запускать триггер?
INSERT, UPDATE и DELETE.

4. Чем AFTER отличается от INSTEAD OF?
AFTER выполняется после операции, INSTEAD OF - вместо неё.

5. Что такое inserted?
Логическая таблица новых строк внутри триггера.

6. Что такое deleted?
Логическая таблица старых строк внутри триггера.

7. Что находится в inserted при UPDATE?
Новые версии изменённых строк.

8. Что находится в deleted при UPDATE?
Старые версии изменённых строк.

9. Почему нельзя считать , что inserted содержит одну строку?
Одна инструкция может добавить или изменить сразу много строк.

10. Как получить старое и новое значение при UPDATE?
Соединить inserted и deleted по первичному ключу.

11. Для чего можно использовать триггер аудита?
Для хранения истории добавлений, изменений и удалений.

12. В каких случаях лучше использовать CHECK , а не триггер?
Для простых проверок значения строки, например age >= 16.

13. Чем триггер отличается от хранимой процедуры?
Триггер запускается автоматически по событию, процедура вызывается явно и может получать параметры.

14. Почему большое количество взаимозависимых триггеров может усложнить систему?
Возникают скрытые действия, вложенные вызовы и зависимости, которые труднее отслеживать.

Практическая часть

Аудит INSERT

```
USE CollegeDB;
GO
CREATE TABLE college.student_audit
(
    id INT IDENTITY(1,1) PRIMARY KEY,
    student_id INT NOT NULL,
    student_name NVARCHAR(100) NOT NULL,
    action_type NVARCHAR(20) NOT NULL,
    action_date DATETIME2 NOT NULL
);
GO
CREATE TRIGGER college.trg_Students_AuditInsert
ON college.students
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO college.student_audit
        (student_id, student_name, action_type, action_date)
    SELECT id, name, N'INSERT', SYSDATETIME()
    FROM inserted;
END;
GO
```

Аудит UPDATE и самостоятельное задание

Записываются только изменения среднего балла, включая переходы между NULL и числом. INSERT ... SELECT обрабатывает все изменённые строки.

```
CREATE TABLE college.student_changes
(
    id INT IDENTITY(1,1) PRIMARY KEY,
    student_id INT NOT NULL,
    old_average_grade DECIMAL(4,2),
    new_average_grade DECIMAL(4,2),
    change_date DATETIME2 NOT NULL
);
GO
CREATE TRIGGER college.trg_Students_AuditUpdate
ON college.students
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO college.student_changes
        (student_id, old_average_grade,
         new_average_grade, change_date)
    SELECT i.id, d.average_grade, i.average_grade, SYSDATETIME()
    FROM inserted i
    JOIN deleted d ON d.id = i.id
    WHERE i.average_grade <> d.average_grade
       OR (i.average_grade IS NULL AND d.average_grade IS NOT NULL)
       OR (i.average_grade IS NOT NULL AND d.average_grade IS NULL);
END;
GO
```

Дополнительное задание: запрет балла больше 5

```
CREATE TRIGGER college.trg_Students_CheckGrade
ON college.students
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted WHERE average_grade > 5)
    BEGIN
        THROW 50002, N'Средний балл не может быть больше 5', 1;
    END;
END;
GO
```

Проверка нескольких строк

На базе после предыдущих заданий INSERT добавляет две строки аудита. UPDATE группы 1 меняет трёх студентов: 4.7 → 4.8, 4.2 → 4.3, 3.6 → 3.7; журнал получает три строки за один запуск триггера. Попытка записать 5.5 выдаёт ошибку 50002 и не меняет итоговый балл Ивана 4.8. Если использовать переменную для одной строки inserted, часть изменений будет потеряна.

```
INSERT INTO college.students (name, age, average_grade, group_id)
VALUES (N'Олег', 18, 4.0, 2), (N'Елена', 19, 4.3, 2);
SELECT * FROM college.student_audit ORDER BY id;

UPDATE college.students
SET average_grade = average_grade + 0.1
WHERE group_id = 1;
SELECT * FROM college.student_changes ORDER BY id;
GO

BEGIN TRY
    UPDATE college.students SET average_grade = 5.5 WHERE id = 1;
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS error_number,
           ERROR_MESSAGE() AS error_message;
END CATCH;
SELECT id, name, average_grade FROM college.students WHERE id = 1;
```